



MARDAS FOLIO
MARKDOWN TO PDF ENGINE

راهنمای کامل

راهنمای Mardas Folio

راهنمای کامل استفاده و مرجع امکانات

این فایل راهنمای کامل نصب، پیکربندی و استفاده از Mardas Folio است. همین سند به عنوان نمونه زنده رندر نیز استفاده می‌شود و جلد، فهرست مطالب، متن ترکیبی فارسی/English، فرمول، کد، نمودار Mermaid، تصویر، جدول، پانویس، شکست صفحه و HTML امن را نمایش می‌دهد.

مؤسسه

Mardas Lab

تاریخ

۱۴۰۵-۰۲-۳۰

نویسندگان

تیم Mardas Folio

(مستندسازی)

Meraj Rastegar

(mragestsars@gmail.com)

وضعیت

Stable

نسخه

2.0.3

درس / دوره

انتشار حرفه‌ای Markdown

کلیدواژه‌ها

Markdown

PDF

فارسی/English

RTL/LTR

MathJax

فهرست مطالب

۱ معرفی

۱-۱ چک لیست نمونه های رندر

۱-۲ قابلیت های اصلی

۲ نصب

۲-۱ پیش نیازها

۲-۲ نصب از سورس

۲-۳ نصب برای توسعه

۲-۴ مستقل برای بسته بندی دسکتاپ Runtime

۲-۵ پیش نمایش نویسنده گی بومی Mardas Folio

۲-۵-۱ ساخت و انتشار Book Project

۲-۶ بسته های نصب دسکتاپ و فایل پشتیبانی

۲-۷ تجربه هدایت شده دسکتاپ و دسترس پذیری

۳ اولین خروجی PDF

۴ روند پیشنهادی کار

۵ درباره Front Matter

۵-۱ فیلدهای رایج

۵-۲ رفتار جلد

۶ زبان و جهت سند

۶-۱ اولویت تعیین جهت

۶-۲ نمونه متن ترکیبی

۶-۳ نمونه smoke تصویری فارسی/RTL

۷ فهرست مطالب

۸ مرجع امکانات Markdown

۸-۱ پاراگراف و تاکید

۸-۲ لیست ها

۸-۳ فهرست وظایف

۸-۴ جدول

۸-۵ نقل قول

۸-۶ انواع Callout

۹	فرمول‌های MathJax
۹-۱	رفع اشکال فرمول‌ها
۱۰	بلوک‌های کد
۱۰-۱	بلوک‌های کد پیشرفته
۱۱	نمودارهای Mermaid
۱۲	تصویر و HTML امن
۱۲-۱	قواعد استفاده از تصویر
۱۲-۲	کد HTML امن
۱۳	امنیت و ورودی‌های قابل اعتماد
۱۳-۱	ناوبری PDF و metadata
۱۳-۲	یکپارچگی ورودی و خروجی
۱۴	صفحه‌بندی و Layout
۱۴-۱	شکست صفحه دستی
۱۴-۲	اندازه صفحه
۱۴-۳	انواع Margin
۱۵	اعمال Watermark
۱۶	برندینگ جلد
۱۷	سیستم Appearance
۱۸	پروفایل‌های کنترل کیفیت انتشار
۱۹	پیکربندی پروژه و Diagnostic‌های ساخت‌یافته
۲۰	حالت کتاب چندفایلی
۲۱	ارجاع متقابل و شماره‌گذاری
۲۲	کتاب‌نامه و استناد
۲۳	کارابی و اسناد بزرگ
۲۴	دسترس‌پذیری و آمادگی آرشیوی
۲۵	روند کار با GUI
۲۶	مرجع CLI
۲۷	اتوماسیون و CI
۲۷-۱	اعتبارسنجی فایل‌های Release
۲۸	رفع اشکال
۲۸-۱	پیدا نشدن Chromium

۲۸-۲ متن فارسی ظاهر مناسبی ندارد

۲۸-۳ تصویرها دیده نمی‌شوند

۲۸-۴ فرمول‌ها به شکل TeX خام دیده می‌شوند

۲۸-۵ نیاز به بررسی Layout

۲۹ بررسی Preflight فایل PDF

۳۰ پانویس

۳۱ چک‌لیست نهایی انتشار

■ فهرست شکل‌ها

شکل ۱ جریان پردازش ارجاع

■ فهرست جدول‌ها

جدول ۱ نوع‌های معنایی ارجاع

■ فهرست معادله‌ها

معادله ۱ $E = mc^2$

■ فهرست کدها

کد ۱ اعتبارسنجی ارجاع

نکته

این راهنما مرجع کامل کاربر و مرجع امکانات Mardas Folio است. هر قابلیت پشتیبانی‌شده با Markdown قابل اجرا آموزش داده می‌شود و همان نمونه‌ها در PDF رسمی هم دیده می‌شوند.

برنامه Mardas Folio ابزاری برای تبدیل Markdown به PDF است که مخصوص سندهای فارسی، انگلیسی و ترکیبی طراحی شده است. هدف پروژه این است که نویسنده بتواند متن را در قالب ساده Markdown بنویسد، اما خروجی نهایی شبیه یک سند PDF مرتب، حرفه‌ای و قابل انتشار باشد.

این پروژه برای گزارش‌های دانشگاهی، مستندات فنی، جزوه‌های آموزشی، راهنماهای نرم‌افزاری، پیش‌نویس‌های پژوهشی، گزارش پروژه و هر سند Markdown که نیاز به خروجی PDF تمیز دارد مناسب است.

TEXT

Markdown -> Structured HTML -> Chromium PDF

در این پروژه متن‌ها مستقیماً روی canvas فایل PDF رسم نمی‌شوند. ابتدا Markdown به HTML ساختاریافته تبدیل می‌شود، سپس CSS چاپی و تنظیمات appearance اعمال می‌شود، فرمول‌ها با MathJax رندر می‌شوند و در مرحله آخر Chromium خروجی PDF را تولید می‌کند. این روش باعث پشتیبانی بهتر از layout چاپی، متن‌های ترکیبی RTL/LTR، فرمول‌های SVG، کدهای هایلایت‌شده، تصویرهای محلی و جدول‌های پیچیده می‌شود.

نکته

این راهنما هم مستند استفاده است و هم نمونه رندر. نسخه PDF همین فایل در پوشه examples/ قرار دارد تا کاربر بتواند خروجی واقعی هر قابلیت مهم را بررسی کند.

پیشنهاد

مستندات کاربر در همین guide نگه داشته می‌شود و صفحه‌های feature/reference موازی حذف شده‌اند تا متن آموزشی و نمونه‌های PDF از هم جدا و ناسازگار نشوند. فایل‌های release، maintenance، security و changelog فقط برای عملیات پروژه در پوشه docs/ باقی مانده‌اند.

چک لیست نمونه‌های رندر

این راهنما عمده‌اً چند نمونه کوچک اما مهم دارد تا کنار راهنمای برنامه، مثل test case تصویری هم عمل کند. وقتی PDF خروجی را بررسی می‌کنید، این موارد را ببینید:

تصویرها، جدول‌ها، بلوک‌های کد و نمودارهای Mermaid دارای caption حالا به صورت بلوک‌های معنایی چاپی نرمال می‌شوند تا توضیح هر بخش در PDF کنار محتوای مربوط به خودش باقی بماند.

بخش نمونه	چیزی که باید در PDF بررسی شود
جلد و metadata	عنوان، زیرعنوان، نویسنده‌ها، خلاصه، نسخه، وضعیت، کلیدواژه و label‌های وابسته به زبان.
فهرست مطالب و outline	شماره‌گذاری تودرتو، لینک‌های داخلی و bookmark‌های PDF viewer که از heading‌های Markdown ساخته می‌شوند.
متن ترکیبی	متن فارسی/English، inline code و شناسه‌ها در یک پاراگراف خوانا بمانند.
فرمول MathJax	فرمول درون‌خطی با متن هماهنگ باشد و فرمول نمایشی وسط‌چین و خوش‌اندازه دیده شود.
بلوک کد	indented، fenced و code block بدون زبان بدون خراب شدن محتوا رندر شوند.
نمودار Mermaid	بلوک کد از نوع <code>flowchart</code> به جای نمایش کد خام، به نمودار SVG تبدیل شود.
تصویر و HTML	تصویر Markdown و تگ امن HTML در PDF دیده شوند.
پانویس و صفحه‌بندی	ارجاع عددی پانویس، لینک بازگشت برای ارجاع‌های تکراری، پانویس چندخطی، شکست صفحه دستی، margin‌ها و شماره صفحه پایدار باشند.
audit فارسی/RTL	نشانه‌گذاری فارسی، عددهای فارسی/لاتین، جدول‌های RTL، عنوان‌های mixed-script در فهرست، caption و back-link پانویس خوانا بمانند.

قابلیت‌های اصلی

قابلیت	توضیح
سند‌های فارسی و انگلیسی	پشتیبانی از <code>lang: fa</code> ، <code>lang: en</code> ، جهت RTL/LTR و متن ترکیبی.
جلد حرفه‌ای	عنوان، زیرعنوان، نویسنده‌ها، خلاصه، لوگو، تاریخ، نسخه، وضعیت، کلیدواژه و metadata آموزشی.
فهرست مطالب و outline	ساخت فهرست چندسطحی و bookmark‌های PDF viewer از heading‌های Markdown.
فرمول MathJax	رندر فرمول‌های درون‌خطی و نمایشی.

قابلیت	توضیح
بلوک کد	هایلایت کدهای fenced و indented با Pygments.
نمودار Mermaid	رندر آفلاین نمودارهای کاربردی <code>graph</code> / <code>flowchart</code> به صورت SVG.
تصویر محلی	جاسازی تصویرهای Markdown و HTML امن به صورت data URI.
کد HTML امن	پاکسازی HTML خام به صورت پیش فرض.
پانویس	پشتیبانی از پانویس‌های چندخطی با محتوای Markdown.
سیستم Appearance	انتخاب جداگانه <code>style</code> ، <code>palette</code> و <code>mode</code> برای شکل سند، رنگ‌بندی و خروجی روشن/تاریک.
اتوماسیون	رابط CLI مناسب برای اسکریپت‌ها و CI.
رابط گرافیکی GUI	رابط گرافیکی محلی برای ویرایش، پیش‌نمایش تقریبی، تنظیم گزینه‌ها و خروجی گرفتن.

نصب

پیش‌نیازها

برای استفاده از پروژه بهتر است این موارد آماده باشند:

- نصب بودن Python نسخه 3.10 یا جدیدتر؛
- محیط مجازی Python؛
- نصب بودن Chromium مربوط به Playwright؛
- فونت مناسب فارسی، ترجیحاً Vazirmatn؛
- نصب بودن Git برای clone کردن پروژه.

■ نصب از سورس

BASH

```
git clone https://github.com/mragetsars/Mardas-Folio.git
cd Mardas-Folio
python -m venv .venv
source .venv/bin/activate
pip install -e .
python -m playwright install chromium
```

در: Windows PowerShell:

POWERSHELL

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -e .
python -m playwright install chromium
```

■ نصب برای توسعه

برای توسعه و اجرای تست‌ها:

BASH

```
pip install -e .[dev]
pytest
```

بعد از نصب، سه رابط پردازشی در دسترس است:

دستور	کاربرد
<code>folio</code>	تبدیل Markdown به PDF از خط فرمان.
<code>folio-gui</code>	اجرای رابط گرافیکی مرورگر محور فعلی.
<code>folio-sidecar</code>	اجرای موتور JSON-RPC نسخه دار روی ورودی/خروجی استاندارد برای برنامه دسکتاپ.

Runtime مستقل برای بسته بندی دسکتاپ

فرایند انتشار می تواند یک runtime قابل حمل بسازد که Python، موتور Mardas، منابع Playwright و Chromium pin شده را داخل خود دارد. روی سیستم مقصد نیازی به نصب Python، pip، Node.js، Git یا Chrome نیست. نسخه 2.0.3 می تواند همین runtime اعتبارسنجی شده را در بسته های بومی Mardas Folio که روی runner همان سیستم عامل ساخته می شوند قرار دهد.

BASH

```
python -m pip install -e '.[desktop]'
python -m playwright install chromium --only-shell
python scripts/build_standalone_runtime.py --clean
```

برای بررسی manifest داخلی، چرخه protocol، مرورگر همراه برنامه و یک رندر واقعی در مسیر Unicode/فارسی:

BASH

```
python scripts/verify_standalone_runtime.py \
  build/standalone-runtime/Mardas-Folio-2.0.3-runtime-windows-x86_64 \
  --render
```

نسخه ۲ runtime manifest فایل های معمولی و symlink های نسبی صریح را ثبت می کند. کنترل های build، staging، archive و provenance لینک امن را حفظ می کنند و target مطلق یا خارج شونده، لینک dangling، cycle، عبور مسیر از symlink و ناهماهنگی inventory را رد می کنند. manifest قدیمی نسخه ۱ فقط برای فهرست فایل های معمولی همچنان قابل بررسی است.

Sidecar هیچ پورت localhost باز نمی کند. هر خط `stdin` یک درخواست JSON-RPC است، `stdout` فقط برای پیام های protocol استفاده می شود و log ها به `stderr` می روند. سقف envelope کد شده درخواست ۶۴ MiB و سقف متن UTF-8 رمزگشایی شده عملیات مستقل هر سند ۸ MiB است؛ فایل های Project Workspace همچنان مرز

سخت‌گیرانه‌تر ۴ MiB دارند. برنامه دسکتاپ باید پیش از رندر، `system.health` و `system.capabilities` را فراخوانی کند.

پیش‌نمایش نویسندگی بومی Mardas Folio

Mardas Folio به‌جای tab مرورگر در یک پنجره بومی Tauri باز می‌شود. Start Center انتخاب فایل بومی، اسناد اخیر، **Quick Export** و انتخاب پوشه پروژه را ارائه می‌کند. نسخه 2.0.3 یک پنل پروژه هوشمند دارد که آخرین پروژه را در session بعدی بازایی می‌کند، فایل‌های پشتیبانی‌شده زیر ریشه `mardas.toml` را نمایش می‌دهد، در محتوای Unicode با جست‌وجوی متنی یا regex محدودشده می‌گردد و هر نتیجه را در خط دقیق editor باز می‌کند.

پنل منابع، فایل‌های محلی BibTeX و CSL JSON تنظیم‌شده در پروژه را index می‌کند، جست‌وجوی عنوان، نویسنده، سال و کلید را ارائه می‌دهد، منبع‌های استفاده‌شده را مشخص می‌کند، خطاهای parse را نشان می‌دهد و citation انتخابی را در محل cursor درج می‌کند. heading‌های پیش‌نمایش نیز metadata معتبر خط منبع را از موتور Markdown پایتون دریافت می‌کنند؛ بنابراین عنوان‌های تکراری میان preview، outline و editor به مقصد صحیح هدایت می‌شوند.

ذخیره سند همچنان conflict-aware است. Engine API نسخه 1.5.0 برای Markdown از `document.read` / `document.save` و برای فایل‌های UTF-8 پشتیبانی‌شده با پسوند `.bib`، `.json`، `.toml`، `.txt`، `.yaml` و `.yml` از `document.read_text` / `document.save_text` استفاده می‌کند. پاسخ خواندن و ذخیره `kind`، `revision` و `read_only` را برمی‌گرداند؛ عملیات پروژه همین metadata را همراه token تعارض SHA-256 نگه می‌دارد. تغییرات اتمیک نوشته می‌شوند و اگر برنامه دیگری فایل را تغییر داده باشد، به‌جای overwrite خاموش متوقف می‌شوند. local storage برای هر سند snapshot محدود recovery نگه می‌دارد و بدون اقدام صریح کاربر چیزی در فایل اصلی ذخیره نمی‌کند. asset‌های واردشده نیز به نوع‌های محلی مجاز زیر پوشه `assets/` سند محدودند.

textarea قبلی با CodeMirror 6 و همان editor adapter تست‌شده جایگزین شده است. bundle قطعی editor و notice‌های آن داخل برنامه قرار دارند؛ بنابراین کل محیط ویرایش بدون CDN یا دانلود dependency در زمان اجرا کاملاً آفلاین کار می‌کند و state مربوط به `recovery`، `conflict`، پروژه و preview همچنان خارج از widget می‌ماند. خروجی PDF مرجع نهایی Mermaid، MathJax، صفحه‌بندی، فونت و layout چاپی باقی می‌ماند. بسته‌های بومی موتور frozen Chromium pin شده را داخل خود دارند، بنابراین کاربر عادی لازم نیست Python، Playwright، Rust یا local server را آماده کند.

ساخت و انتشار Book Project

در Start Center گزینه **پروژه کتاب جدید** را انتخاب کنید، پوشه مادر و عنوان کتاب را مشخص کنید. Mardas Folio یک پروژه مستقل شامل `mardas.toml`، فصل نخست Markdown، پوشه‌های مشترک asset و bibliography و پوشه خروجی `dist` می‌سازد.

پنل کتاب مسیر عادی انتشار را ارائه می‌کند:

1. فصل تازه بسازید یا فصل موجود را تکثیر کنید.

2. هر فصل را در editor چن‌دنب باز کنید.

3. ترتیب فصل‌ها را با دکمه‌های جابه‌جایی یا drag-and-drop تغییر دهید.

4. فصل را بدون حذف فایل Markdown از فهرست کتاب خارج کنید.

5. همه فصل‌ها و ارجاع‌های پروژه را اعتبارسنجی کنید.

6. پیش‌نمایش کامل کتاب را ببینید.

7. یک PDF نهایی را با انتخاب‌گر بومی مسیر خروجی بسازید.

تغییر ترتیب فصل‌ها با revision فعلی تنظیمات پروژه محافظت می‌شود. اگر `mardas.toml` بیرون از برنامه تغییر کند، عملیات به جای overwrite خاموش متوقف می‌شود. پیش‌نمایش و خروجی کامل کتاب jobهای قابل لغو Sidecar هستند و از همان موتور Book Mode رابط خط فرمان استفاده می‌کنند.

■ بسته‌های نصب دسکتاپ و فایل پشتیبانی

→ build انتشار credential نسخه 2.0.3 فقط در صورت قراردادادن کلید عمومی نگهدارنده می‌تواند بخش **Settings** را فعال کند. بررسی update فقط با اقدام کاربر انجام می‌شود، درخواست از مرز بومی Rust عبور می‌کند، endpoint باید HTTPS باشد و نصب فقط برای payload دارای امضای معتبر Tauri انجام می‌شود. build توسعه‌ای بدون آن کلید updater فعال ندارد. tag نسخه فقط پس از موفقیت jobهای Windows، macOS، Linux، checksum، SBOM، metadata، بروزرسانی و attestation می‌تواند یک GitHub Draft Release بسازد و هرگز خودکار منتشر نمی‌کند. تست‌های repository تضمین code signing و ویندوز یا Developer ID/notarization مک نیستند؛ مدرک آن‌ها باید از اجرای واقعی انتشار credential بازبینی شود.

فرایند انتشار اکنون Mardas Folio را به‌عنوان یک برنامه دسکتاپ قابل نصب در نظر می‌گیرد، نه ابزاری که کاربر نهایی مجبور باشد محیط توسعه آن را آماده کند. ماتریس انتشار برای ساخت Setup و بسته قابل حمل Windows، فایل‌های DMG مخصوص معماری‌های macOS و بسته‌های AppImage و Debian برای Linux پیکربندی شده است. مقصدهای پشتیبانی‌شده شامل 11 Windows روی x86-64 (یا Windows Server 2019 و جدیدتر)، macOS 14 و جدیدتر روی Apple Silicon یا Intel، و بسته‌های Linux روی x86-64 با مبنای آزمون Ubuntu 22.04 هستند. AppImage ممکن است روی توزیع‌های جدیدتر و سازگار با glibc نیز اجرا شود، اما آن‌ها بیرون از ماتریس پذیرش رسمی‌اند. موفقیت هر خروجی باید از job همان release تأیید شود. برنامه عادی Sidecar و Chromium pin شده‌ی رندر را همراه خود دارد؛ کاربر نهایی به Python، Node.js، Rust، Git، یا نصب جداگانه Playwright نیاز ندارد.

از مسیر **Help** → **ذخیره بسته پشتیبانی** می‌توان یک ZIP تشخیصی ساخت که نسخه Mardas، وضعیت runtime و در دسترس بودن renderer را ثبت می‌کند. این بسته عمداً محتوای سندها، مسیر سندها، متغیرهای محیطی و مسیر پوشه home کاربر را در خود قرار نمی‌دهد.

زیرساخت metadata برای بروزرسانی امضاشده آماده است، اما buildی که کلید عمومی نگه‌دارنده را دریافت نکرده باشد بروزرسانی عمومی را فعال نمی‌کند. کلید خصوصی بروزرسانی نباید در سند، پروژه، support bundle یا repository ذخیره شود.

تجربه هدایت‌شده دسکتاپ و دسترس‌پذیری

در نخستین اجرا، Mardas Folio به‌جای نمایش مستقیم کنترل‌های فنی، یک راهنمای شروع کوتاه نشان می‌دهد. این راهنما تفاوت سند و Book Project، کارکرد آفلاین، ایمنی recovery و میانبرهای اصلی را توضیح می‌دهد. کاربر می‌تواند آن را رد کند و بعداً از تنظیمات دوباره اجرا کند.

Start Center قالب‌های محلی برای سند خالی، گزارش، متن دانشگاهی و سند فنی دارد. انتخاب قالب، Markdown، قابل‌ویرایش را کاملاً محلی می‌سازد و در زمان اجرا چیزی از اینترنت دانلود نمی‌شود. تنظیمات امکان جست‌وجوی گزینه‌ها و انتخاب ظاهر system/light/dark، مقیاس محتوا، کاهش حرکت، پیش‌نمایش خودکار و زبان رابط را فراهم می‌کند.

راهنما مسیرهای سند، کتاب، خروجی، recovery و حریم خصوصی را توضیح می‌دهد؛ `Ctrl/Cmd+Shift+P` Command Palette و `F1` راهنما را باز می‌کند.

دسترس‌پذیری صفحه‌کلید بخشی از قرارداد دسکتاپ است: skip link کاربر را به محتوای اصلی می‌رساند، دکمه‌های فقط‌آیکون نام قابل‌دسترسی دارند، dialogها هنگام بازبودن focus را داخل خود نگه می‌دارند و پس از بسته‌شدن focus قبلی را برمی‌گردانند، dialogهای امن با `Escape` بسته می‌شوند، پیام‌ها از ARIA live region استفاده می‌کنند و focus قابل‌مشاهده برای کاربر صفحه‌کلید حفظ می‌شود. CI در هر build دسکتاپ audit ساختاری accessibility را اجرا می‌کند و smoke واقعی مرورگر، onboarding، templateها، preferenceها، command navigation و رفتار RTL را بررسی می‌کند.

اولین خروجی PDF

تبدیل ساده:

BASH

```
folio input.md -o output.pdf
```

تبدیل همراه با فهرست مطالب و ظاهر GitHub-style:

BASH

```
folio input.md -o output.pdf --toc --style modern --palette emerald --mode light
```

تولید خروجی طولانی با رفتار شبیه کتاب:

BASH

```
folio input.md -o output.pdf \  
  --toc \  
  --toc-depth 4 \  
  --toc-page-break \  
  --h1-page-break \  
  --style textbook \  
  --palette slate \  
  --mode light
```

ذخیره HTML میانی برای بررسی و رفع اشکال:

BASH

```
folio input.md -o output.pdf --debug-html output.html
```

نمایش صریح progress bar در ترمینال:

BASH

```
folio input.md -o output.pdf --progress on
```

حالت پیش‌فرض `--progress auto` فقط وقتی progress bar را نشان می‌دهد که دستور در ترمینال تعاملی اجرا شود. برای اسکرپت‌های ساکت‌تر می‌توانید از `--progress off` استفاده کنید.

روند پیشنهادی کار

برای رسیدن به خروجی تمیز، این روند پیشنهاد می‌شود:

قواعد صفحه‌بندی PDF حالا تلاش می‌کنند عنوان‌ها را کنار اولین بلوک بعدی نگه دارند، از تک‌افتادن خط‌های پاراگراف جلوگیری کنند و در صورت طولانی بودن بلوک کد یا جدول، آن‌ها را تمیزتر به صفحه بعد ادامه دهند تا فضای سفید بزرگ ایجاد نشود.

1. محتوای سند را در Markdown بنویسید.

2. در ابتدای فایل، front matter شامل عنوان، نویسنده، زبان، جهت و metadata جلد را اضافه کنید.

3. یک خروجی اولیه با `--toc` و appearance مناسب بگیرید.

4. جلد، فهرست مطالب، صفحه‌های دارای فرمول، صفحه‌های دارای کد، تصویرها و شماره صفحه را بررسی کنید.

5. اگر layout نیاز به بررسی داشت، خروجی `--debug-html` بگیرید و HTML تولیدشده را در مرورگر ببینید.

6. اصلاحات نهایی را در Markdown انجام دهید و PDF را دوباره بسازید.

برای سندهای مهم، بررسی انسانی خروجی نهایی ضروری است. تست خودکار بسیاری از خطاها را می‌گیرد، اما تایپوگرافی، شکست صفحه و فاصله‌ها باید دیده شوند.

درباره Front Matter

بخش Front matter یک بخش YAML اختیاری در ابتدای فایل Markdown است. این بخش جلد، metadata فایل PDF، زبان سند، جهت سند و چند فیلد مخصوص گزارش‌های رسمی یا آموزشی را کنترل می‌کند.

```

---
title: "گزارش فنی من"
subtitle: "Markdown ساخته شده با PDF یک سند"
authors:
  - name: "Mardas"
    email: "mragetsars@gmail.com"
    affiliation: "Mardas Lab"
    role: "Author"
summary: |
  این متن روی جلد نمایش داده می شود.
  متن چند خطی پشتیبانی می شود
institution: "نام دانشگاه یا سازمان"
department: "نام دانشکده یا دپارتمان"
course: "نام درس یا پروژه"
supervisor: "نام استاد یا راهنما"
date: "۱۴۰۵-۰۲-۳۰"
version: "2.0.3"
status: "Draft"
keywords: [Markdown, PDF, RTL, MathJax]
cover_label: "گزارش فنی"
branding:
  mode: full
brand:
  name: "آزمایشگاه پژوهشی Acme"
  logo: "assets/acme-logo.svg"
  footer: "گزارش فنی داخلی"
lang: fa
dir: rtl
---

```

فیلدهای رایج

کاربرد	فیلد
عنوان جلد و title در metadata فایل PDF.	<code>title</code>
متن اختیاری زیر عنوان اصلی.	<code>subtitle</code>
نویسنده ساده و تکمقداری.	<code>author</code>
فهرست نویسندگان با <code>name</code> ، <code>email</code> ، <code>affiliation</code> و <code>role</code> .	<code>authors</code>

کاربرد	فیلد
خلاصه روی جلد و subject در metadata.	<code>summary</code> / <code>description</code>
اطلاعات دانشگاهی یا سازمانی.	<code>institution</code> , <code>department</code> , <code>course</code>
فیلدهای اختیاری برای گزارش‌های آموزشی.	<code>supervisor</code> , <code>group</code> , <code>student_id</code>
کارت‌های وضعیت سند روی جلد.	<code>date</code> , <code>version</code> , <code>status</code>
کلیدواژه‌های جلد و metadata فایل PDF.	<code>keywords</code> / <code>tags</code>
برچسب کوچک بالای عنوان جلد.	<code>cover_label</code>
حالت برندینگ جلد: <code>off</code> , <code>subtle</code> یا <code>full</code> . مقدار پیش‌فرض <code>off</code> است.	<code>branding.mode</code>
برند سازمانی اختیاری که در صورت فعال بودن branding روی جلد می‌آید.	<code>brand.name</code> , <code>brand.logo</code> , <code>brand.footer</code>
مسیر لوگوی سفارشی قدیمی/ساده نسبت به فایل Markdown. فایل باید تصویر معمولی پشتیبانی‌شده و داخل ریشه سند باشد. برای سندهای جدید <code>brand.logo</code> تمیزتر است.	<code>cover_logo</code> / <code>logo</code>
زبان داخلی رابط سند، معمولاً <code>fa</code> یا <code>en</code> .	<code>lang</code>
جهت پوسته سند: <code>auto</code> , <code>ltr</code> یا <code>rtl</code> .	<code>dir</code>

■ رفتار جلد

جلد جدا از محتوای اصلی رندر می‌شود. نتیجه این تصمیم چنین است:

- جلد جزو شماره صفحات محتوایی حساب نمی‌شود؛
- شماره‌گذاری footer از صفحه بعد از جلد شروع می‌شود؛
- طرح watermark فقط روی صفحات محتوایی اعمال می‌شود؛
- style خروجی می‌تواند برای جلد پس‌زمینه تمام‌صفحه داشته باشد؛
- در صورت نیاز، جلد کاملاً قابل حذف است.

حذف جلد:

BASH

```
folio input.md -o output.pdf --no-cover
```

جلد به صورت پیش‌فرض بدون برندینگ خروجی می‌گیرد تا سند عادی شبیه تبلیغ ابزار نباشد. برندینگ کامل را فقط وقتی فعال کنید که واقعاً لازم است:

BASH

```
folio input.md -o output.pdf --branding full --brand-name "Acme Research Lab"
```

استفاده از لوگوی برند سفارشی:

BASH

```
folio input.md -o output.pdf --branding full --brand-logo  
./assets/logo.png
```

حفظ جلد بدون نمایش لوگو:

BASH

```
folio input.md -o output.pdf --no-cover-logo
```

برای یادداشت کوچک تولیدشده با ابزار از `branding.mode: subtle` استفاده کنید. راهنماهای رسمی پروژه چون خود Mardas Folio را مستند می‌کنند، به صورت صریح `branding.mode: full` دارند.

زبان و جهت سند

کنترل جهت یکی از مهم‌ترین بخش‌های تولید PDF فارسی/انگلیسی است. در Mardas Folio زبان سند و جهت سند از هم جدا در نظر گرفته شده‌اند.

باعث ایجاد پوسته فارسی/RTL، لیبل‌های فارسی جلد، عنوان فارسی callout ها و عنوان `lang: fa` می‌شود. **فهرست مطالب**

باعث ایجاد پوسته انگلیسی/LTR، لیبل‌های انگلیسی جلد، عنوان انگلیسی callout ها و عنوان `lang: en` می‌شود. **Table of Contents**

اولویت تعیین جهت

جهت سند با این ترتیب مشخص می‌شود:

1. گزینه خط فرمان `--dir rtl|ltr|auto`

2. فیلدهای front matter مثل `dir`، `direction`، `text_direction` یا `document_direction`

3. پیش‌فرض به‌دست‌آمده از `lang`

4. تشخیص خودکار از متن Markdown

■ نمونه متن ترکیبی

در متن انگلیسی می‌توان واژه‌های فارسی مثل راست به چپ، فونت فارسی و گزارش فنی را آورد بدون اینکه جمله انگلیسی به هم بریزد.

در متن فارسی نیز می‌توان شناسه‌های English مثل `Playwright`، `MathJax`، `GitHub Actions`، `PDF` و `RTL/LTR` را داخل همان پاراگراف استفاده کرد.

همچنین Inline code هم پایدار می‌ماند: `folio input.md -o output.pdf --toc`.

■ نمونه smoke تصویری فارسی/RTL

این نمونه کوچک عمداً داخل guide مانده است، چون guide هم راهنمای کاربر است و هم test case زنده ^۱ renderer. این بخش نشانه‌گذاری فارسی، نام‌های لاتین، عدد فارسی، caption جدول، و سلول‌های mixed-direction را در PDF رسمی نگه می‌دارد.

^۱ این نمونه زنده عمداً نشانه‌گذاری فارسی، شناسه‌های لاتین، عددهای فارسی، caption جدول و پانویس‌های محلی همان صفحه را داخل راهنمای رسمی فارسی نگه می‌دارد.

آیا خروجی PDF برای `version 2.0.3` و شماره ۱۴۰۵ پایدار است؟ پاسخ: بله؛ جدول زیر باید hook‌های RTL، mixed-script و mixed-number را فعال کند.

جدول ۱۲. نمونه جدول فارسی/RTL با عددهای ترکیبی.		
بخش نمونه	مقدار	انتظار در PDF
شماره فارسی	۱۴۰۵	عدد فارسی کنار متن RTL پایدار بماند.
نسخه فنی	version 2.0.3 و ۱.۹.۹	عددهای Latin/Persian در یک سلول خوانا بمانند.
شناسه انگلیسی	<code>MathJax</code> ، <code>TOC</code> ، <code>PDF</code>	identifierهای English داخل جدول فارسی جابه‌جا نشوند.

همین بخش چک‌لیست فارسی/RTL برای شناسه‌های ترکیبی، سبک عددها، جدول‌های RTL، caption و لینک‌های بازگشت پانویس است. راهنمای انگلیسی هم یک نمونه smoke کوچک از همین رفتارها دارد.

فهرست مطالب

فعال کردن فهرست مطالب:

BASH

```
folio input.md -o output.pdf --toc
```

تعیین عمق فهرست:

BASH

```
folio input.md -o output.pdf --toc --toc-depth 3
```

شروع متن اصلی از صفحه جدید بعد از فهرست:

BASH

```
folio input.md -o output.pdf --toc --toc-page-break
```

شروع هر heading سطح اول از صفحه جدید:

BASH

```
folio input.md -o output.pdf --h1-page-break
```

فهرست مطالب از heading های Markdown ساخته می شود و فرمول های درون خطی داخل heading ها مثل $E = mc^2$ و ϵ را خوانا نگه می دارد.

لینک های فهرست مطالب چاپی و bookmark های PDF viewer از مقصدهای ثابت همان heading ها استفاده می کنند؛ بنابراین کلیک روی هر لایه ناوبری کاربر را به عنوان واقعی در متن سند می برد، نه به ردیف مشابه داخل خود فهرست مطالب.

مرجع امکانات Markdown

پاراگراف و تاکید

پاراگراف‌های Markdown، **متن bold**، متن *italic*، `inline code`، لینک، لیست مرتب، لیست نامرتب، task list و نقل قول پشتیبانی می‌شوند.

لیست‌ها

- نوشتن محتوای اصلی با Markdown.
- افزودن front matter برای metadata.
- انتخاب appearance مناسب.
- بررسی خروجی نهایی PDF.

1. نصب پکیج.

2. نصب Chromium.

3. اجرای CLI.

4. بررسی خروجی.

فهرست وظایف

- ورودی Markdown ☒
- تایپوگرافی فارسی/English ☒
- رندر MathJax ☒
- هایلایت کد ☒
- بازبینی انسانی نهایی ☐

جدول

قابلیت	وضعیت	توضیح
متن ترکیبی RTL/LTR	بله	پاراگراف، heading، لیست و سلول جدول با direction-aware styling رندر می‌شوند.
تصویر محلی	بله	تصویرهای Markdown و HTML امن در صورت امکان embed می‌شوند.
فرمول MathJax	بله	فرمول درون‌خطی و نمایشی اندازه‌گذاری جدا دارند.
هایلایت کد	بله	برای fenced و indented code block از Pygments استفاده می‌شود.
نمودار Mermaid	بله	fence های کاربردی <code>flowchart</code> و <code>graph</code> به نمودار SVG تبدیل می‌شوند.
پانویس	بله	پانویس چندخطی با Markdown داخلی پشتیبانی می‌شود.
کد HTML خام امن	بله	tag ها و event handler های خطرناک حذف می‌شوند.

نقل قول

کیفیت خروجی PDF فقط تبدیل متن نیست. تایپوگرافی، line height، کنتراست، صفحه‌بندی، تصویرها، فرمول‌ها و قابل پیش‌بینی بودن رندر اهمیت دارند.

انواع Callout

انواع Callout ها از markerهای مشابه GitHub استفاده می‌کنند و عنوان آن‌ها با توجه به زبان سند ترجمه می‌شود.

نکته

برای نکته‌های توضیحی که باید از متن اصلی جدا دیده شوند، از callout استفاده کنید.

پیشنهاد

وقتی لازم است HTML دقیق ارسال شده به Chromium را ببینید، از `debug-html output.html--` استفاده کنید.

گزینه --unsafe-html فقط برای فایل‌های محلی قابل اعتماد مناسب است، چون sanitizer داخلی را غیرفعال می‌کند.

فرمول‌های MathJax

فرمول‌های درون‌خطی باید با ارتفاع خط اطراف هماهنگ باشند: $E = mc^2$ ، $T = 500$ و $\Sigma = I \cdot \epsilon$ باید طبیعی وسط جمله قرار بگیرند.

فرمول نمایشی فضای بیشتری می‌گیرد:

$$\int_{-\infty}^{\infty} e^{-x^2} dx = \sqrt{\pi}$$

نمونه ماتریسی:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad \det(A) = -2$$

نمونه: aligned

$$\text{precision} = \frac{TP}{TP + FP}$$

$$\text{recall} = \frac{TP}{TP + FN}$$

رفع اشکال فرمول‌ها

اگر فرمول‌ها به شکل TeX خام دیده شدند:

- مطمئن شوید `--no-mathjax` استفاده نشده باشد؛
- هشدارهای ترمینال را بررسی کنید؛
- برای سندهای خیلی بزرگ مقدار `timeout` مرورگر را افزایش دهید؛
- یک فایل کوچکتر بسازید تا فرمول مشکل‌دار را جدا کنید.

بلوک‌های کد

بلوک‌های کد fenced از برجسته‌سازی نحوی و برجسب زبان پشتیبانی می‌کنند.

PYTHON

```
from dataclasses import dataclass

@dataclass
class Document:
    title: str
    lang: str = "fa"

def render_message(doc: Document) -> str:
    return f"Rendering {doc.title} as PDF"

print(render_message(Document("راهنمای Mardas")))
```

JAVASCRIPT

```
const items = ["Markdown", "Persian", "English", "MathJax", "PDF"];
const message = items.map((item, index) => `${index + 1}.
${item}`).join("\n");
console.log(message);
```

بلوک کد fenced بدون زبان هم معتبر است:

TEXT

این بلوک زبان مشخصی ندارد.
با این حال باید بدون خطا رندر شود.

همچنین indented code block نیز پشتیبانی می‌شود:

TEXT

```
SELECT title, lang, version
FROM documents
WHERE renderer = 'mardas-folio';
```

توجه کنید که inline code در برابر پردازش فرمول و پانویس محافظت می‌شود. یعنی `$x$` و `[^note]` وقتی داخل backtick باشند، به شکل literal باقی می‌مانند.

■ بلوک‌های کد پیشرفته

برای آموزش، بازبینی کد یا مستندات فنی، بلوک‌های کد می‌توانند عنوان فایل، شماره خط و خطوط برجسته‌شده داشته باشند.

MARKDOWN

```
```python title="renderer.py" {2,5-6} linenos
def convert(markdown: str) -> bytes:
 html = render_markdown(markdown)
 pdf = render_pdf(html)
 metadata = inspect_pdf(pdf)
 log_export(metadata)
 return pdf
```
```

بخش `{2,5-6}` یعنی: خط ۲ و بازه‌ی شامل خط‌های ۵ تا ۶ برجسته شوند. شماره‌های highlight بر اساس ردیف‌های داخل همان snippet حساب می‌شوند، مگر اینکه برای شماره‌گذاری فایل اصلی از `linenostart` استفاده شده باشد.

نمونه بالا در PDF به شکل یک بلوک کد با caption سفارشی، شماره خط و highlight رندر می‌شود:

renderer.py

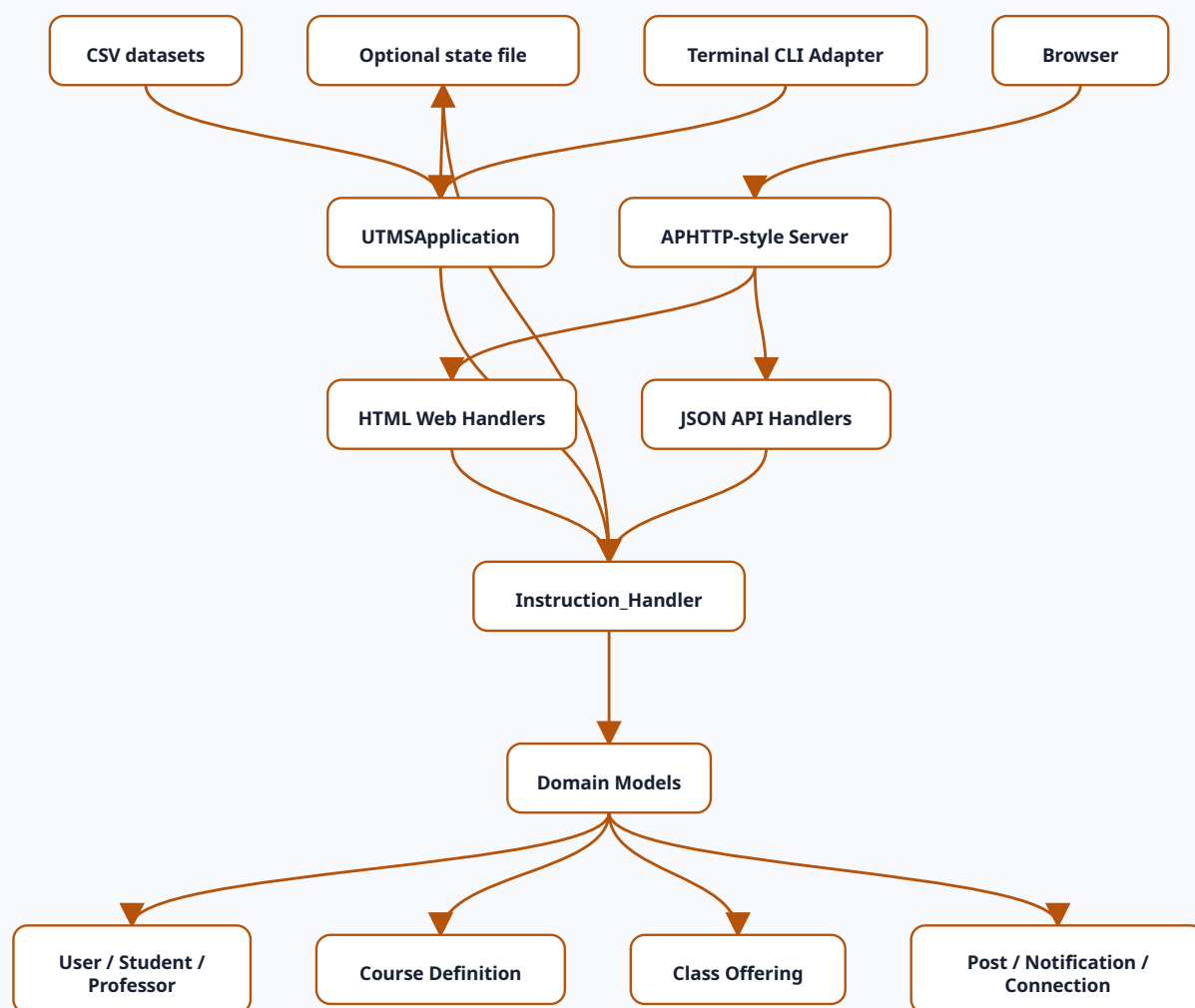
```
1 def convert(markdown: str) -> bytes:
2     html = render_markdown(markdown)
3     pdf = render_pdf(html)
4     metadata = inspect_pdf(pdf)
5     log_export(metadata)
6     return pdf
```

نمودارهای Mermaid

بلوک‌های Mermaid برای flowchart به صورت SVG در HTML میانی رندر می‌شوند و بعد داخل PDF قرار می‌گیرند.

تمرکز renderer فعلاً روی زیرمجموعه رایج در مستندات پروژه است: flowchart / graph ، جهت‌های TD ، TB ، RL ، LR ، BT ، nodeهای مستطیلی، گرد، دایره‌ای و لوزی، edgeهای ساده، dotted، ضخیم و edgeهای دارای label.

نکته مهم در خروجی PDF این است که بلوک زیر باید به شکل نمودار دیده شود، نه کد خام:



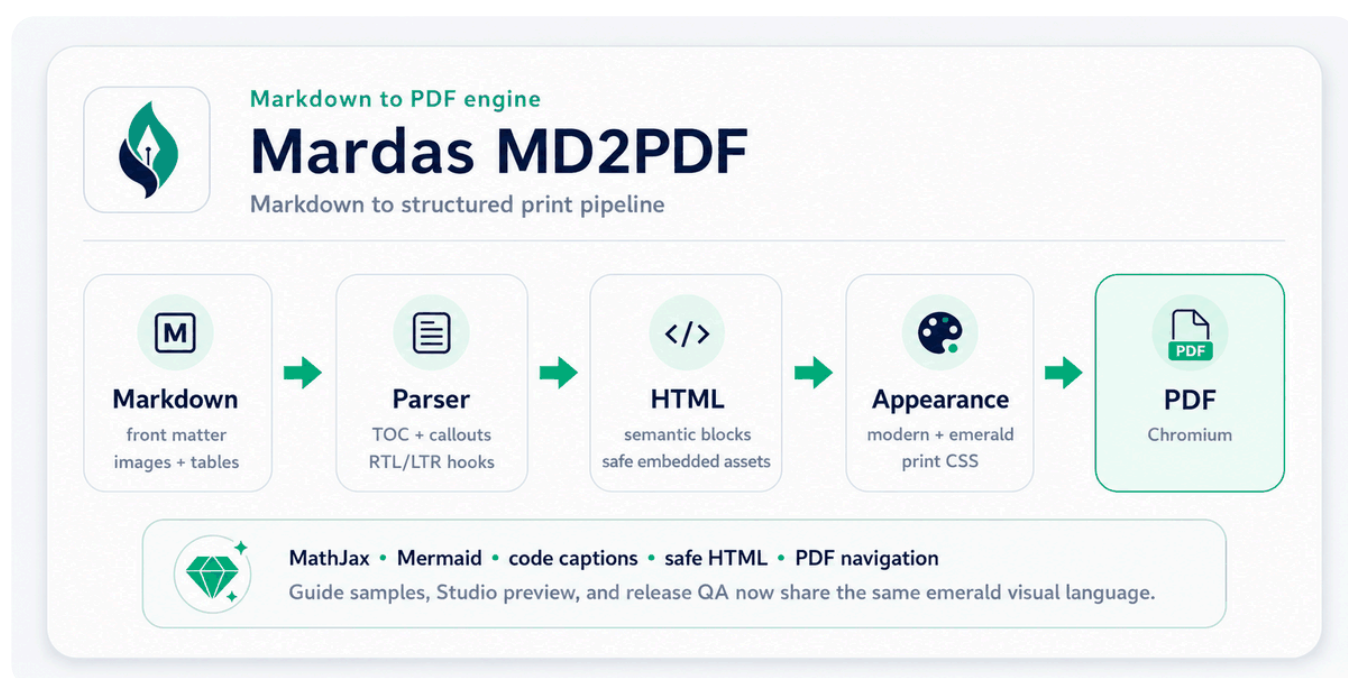
این نمونه کوچکتر برای بررسی label روی edge و شکل node ها مفید است:



اگر یک بلوک Mermaid از syntax های پیشرفته خارج از این زیرمجموعه استفاده کند، بهتر است تا زمان اضافه شدن آن قابلیت، یک تصویر fallback کوچک از همان نمودار کنار Markdown نگه داشته شود. برای معماری های معمول، data flow و نمودارهای آموزشی، renderer داخلی کافی است و به اینترنت نیاز ندارد.

تصویر و HTML امن

مسیر تصویرها نسبت به فایل Markdown resolve می شود. تصویرهای محلی اگر حجم مناسبی داشته باشند داخل HTML/PDF نهایی embed می شوند.



شکل ۱. نمای کلی معماری.

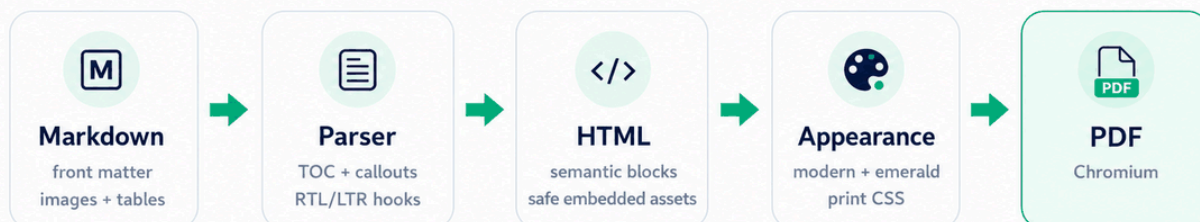
وقتی اندازه دهی دقیق تر لازم است، می توان از HTML امن استفاده کرد. برای اینکه نمونه HTML همان زبان بصری شکل بالا را حفظ کند، در اینجا هم از همان بنر/نمودار اولیه استفاده می شود و دیگر از لوگوی خام استفاده نمی کنیم:



Markdown to PDF engine

Mardas MD2PDF

Markdown to structured print pipeline



MathJax • Mermaid • code captions • safe HTML • PDF navigation

Guide samples, Studio preview, and release QA now share the same emerald visual language.

■ قواعد استفاده از تصویر

- برای خروجی پایدار PDF، تصویرهای محلی بهتر از تصویرهای remote هستند.
- قبل از embed کردن، حجم تصویرها را منطقی نگه دارید.
- مسیر تصویر را نسبت به محل فایل Markdown بنویسید.
- مسیر تصویر را داخل پوشه همان سند نگه دارید. مسیرهای absolute، URLهای `file:`، خروج از پوشه با الگوهایی مثل `../secret.png` و fallback به `current working directory` به صورت پیش فرض block می شوند.
- اگر یک تصویر محلی به شکل امن embed نشود، با یک placeholder شفاف جایگزین می شود تا Chromium آن را از طریق `<base>` سند load نکند.
- در GUI، فایل ها یا پوشه تصویر را قبل از export attach کنید.
- تصویرهای خیلی بزرگ block می شوند و renderer برای جلوگیری از مصرف زیاد حافظه هشدار می دهد.

■ کد HTML امن

کد HTML خام به صورت پیش فرض sanitize می شود. sanitizer عناصر مناسب سند مثل `<div>`، `<span>`، `<table>`، `<figure>` و `<img>` را نگه می دارد، اما محتوای فعال یا خطرناک مثل `script`، `event handler`، `iframe`، `form`، `stylesheet` خارجی، URLهای `file:`، URL scheme نامن و `data:` image های غیر-raster را حذف می کند.

image **data:** امن فقط برای format های raster رایج مثل PNG، JPEG، GIF، WebP، BMP و AVIF مجاز است. data URL از نوع SVG در HTML امن block می‌شود؛ برای نمودارها بهتر است از فایل محلی قابل اعتماد یا بلوک Mermaid استفاده کنید.

غیرفعال کردن sanitizer فقط برای فایل‌های قابل اعتماد:

BASH

```
folio input.md -o output.pdf --unsafe-html
```

امنیت و ورودی‌های قابل اعتماد

Mardas Folio یک ابزار انتشار محلی است. فایل Markdown، assetهای attach شده در GUI، لوگوی جلد، watermark و HTML خام را محتوای نویسنده و قابل اعتماد در نظر بگیرید؛ مگر اینکه renderer را داخل محیط جداشده اجرا کنید.

رندر پیش‌فرض یک مرز محافظه‌کارانه برای فایل‌ها نگه می‌دارد: تصویرهای محلی و لوگوهای branding در front matter نسبت به محل فایل Markdown resolve می‌شوند، باید فایل تصویر معمولی پشتیبانی‌شده داخل ریشه سند باشند و پس از کنترل symlink و اندازه به `data:` URL تبدیل می‌شوند. مسیر خارج از پوشه سند، فایل نامعتبر یا asset غیرقابل embed رد یا block می‌شود. لینک نسبی فایل سیستم نیز به `file:` وابسته به سیستم تولیدکننده تبدیل نمی‌شود. HTML خام هم به صورت پیش‌فرض sanitize می‌شود، مگر اینکه `--unsafe-html` را فعال کنید.

حالت sandbox کرومیوم با `--chromium-sandbox` کنترل می‌شود:

| حالت | رفتار |
|-------------------|--|
| <code>auto</code> | برای کاربر عادی sandbox را روشن نگه می‌دارد و فقط هنگام اجرای root آن را خاموش می‌کند. |
| <code>on</code> | همیشه sandbox کرومیوم را درخواست می‌کند. |
| <code>off</code> | گزینه <code>--no-sandbox</code> را به Chromium می‌دهد؛ فقط در containerهای قابل اعتماد یا CI جداشده استفاده شود. |

برای سندهای نامطمئن، خروجی را داخل container یا محیط disposable بسازید و `--unsafe-html` را خاموش نگه دارید. جزئیات کامل در `../SECURITY.md` آمده است.

■ ناوبری PDF و metadata

PDFهای تولیدشده metadata استاندارد سند را از front matter می‌گیرند و از headingهای Markdown یک outline برای PDF viewer می‌سازند. فهرست مطالب چاپی همچنان داخل متن سند دیده می‌شود، اما outline کنار PDF به خواننده کمک می‌کند سریع بین بخش‌ها جابه‌جا شود.

وقتی فهرست مطالب چاپی هم لازم دارید از `--toc` استفاده کنید. outline از همان مجموعه heading ساخته می‌شود تا فهرست چاپی و bookmarkهای PDF با هم هماهنگ بمانند.

■ یکپارچگی ورودی و خروجی

front matter خراب، بازگشتی، بیش‌ازحد عمیق یا بزرگ با diagnostic کنترل‌شده رد می‌شود و فایل UTF-8 دارای BOM نیز درست خوانده می‌شود. CLI اجازه نمی‌دهد Markdown ورودی، PDF خروجی یا debug HTML به یک فایل واقعی واحد اشاره کنند و خروجی موفق PDF/debug را به شکل atomic جایگزین می‌کند تا خطای نوشتن، artifact سالم قبلی را truncate نکند.

لینک نسبی به فایل سیستم محلی به صورت متن خوانا باقی می‌ماند، اما به لینک `file:` وابسته به سیستم تولیدکننده تبدیل نمی‌شود. شناسه‌های دستی و تولیدشده heading نیز یکتا می‌شوند تا مقصدهای TOC و PDF مبهم نباشند.

صفحه‌بندی و Layout

■ شکست صفحه دستی

برای شکست صفحه دستی می‌توانید HTML امن زیر را قرار دهید:

HTML

```
<div class="md2pdf-page-break"></div>
```

■ اندازه صفحه

استفاده از اندازه نام‌دار:

BASH

```
folio input.md -o output.pdf --page-size A4
```

استفاده از حالت: landscape

BASH

```
folio input.md -o output.pdf --page-size "A4 landscape"
```

استفاده از ابعاد دقیق:

BASH

```
folio input.md -o output.pdf --page-size "210mm 297mm"
```

اندازه‌های نام‌گذاری شده A0-A6، B0-B6، Letter، Legal، Tabloid و Ledger به ابعاد صریح Chromium تبدیل می‌شوند تا format‌های نام‌گذاری شده پشتیبانی نشده بی‌صدا به A4 برگردند. هر ضلع اندازه سفارشی باید بین ۱۰ تا ۵۰۰ میلی‌متر باشد. مقدار ناشناخته یا خارج از محدوده زودتر خطا می‌دهد و Studio نیز همان validation را با خطای ساختاریافته `invalid_page_size` به کار می‌گیرد.

انواع Margin

BASH

```
folio input.md -o output.pdf \  
--margin-top 18mm \  
--margin-bottom 18mm \  
--margin-x 16mm
```

اعمال Watermark

اعمال Watermark متنی:

BASH

```
folio input.md -o output.pdf --watermark "DRAFT"
```

اعمال Watermark تصویری:

BASH

```
folio input.md -o output.pdf \  
--watermark-image ./src/mardas_folio/assets/mardas-folio-logo.png \  
--watermark-opacity 0.05 \  
--watermark-width 95mm
```


طرح Watermark فقط روی صفحات محتوایی اعمال می‌شود و روی جلد قرار نمی‌گیرد.

برندینگ جلد

خروجی‌های PDF به صورت پیش‌فرض بدون برندینگ ساخته می‌شوند. این رفتار باعث می‌شود گزارش، جزوه یا سند کاری کاربر شبیه تبلیغ محصول نباشد و مالکیت بصری سند برای خود کاربر بماند.

YAML

```
branding:
  mode: off
```

حالت‌های موجود:

حالت	کاربرد
off	پیش‌فرض برای گزارش‌ها و جزوه‌های عادی.
subtle	یادداشت کوچک generated-with برای پیش‌نویس‌های غیررسمی.
full	برندینگ عمدی پروژه یا سازمان.

نمونه برند سازمانی:

YAML

```
branding:
  mode: full
brand:
  name: "Acme Research Lab"
  logo: "assets/acme.png"
  footer: "Internal Technical Report"
```

معادل CLI:

```
folio input.md -o output.pdf \  
  --branding full \  
  --brand-name "Acme Research Lab" \  
  --brand-logo assets/acme.png \  
  --brand-footer "Internal Technical Report"
```

راهنماهای فارسی و انگلیسی پروژه چون نمونه رسمی Mardas Folio هستند، به شکل صریح از `branding.mode: full` استفاده می‌کنند.

سیستم Appearance

برنامه Mardas Folio به جای چند سیستم موازی، یک مدل ظاهر دارد: `style` شکل و layout سند را کنترل می‌کند، `palette` رنگ‌های accent را تعیین می‌کند و `mode` خروجی روشن یا تاریک را انتخاب می‌کند.

کاربرد پیشنهادی	Style
مستندات عمومی، گزارش نرم‌افزاری و راهنمای محصول.	<code>modern</code>
مستندات پروژه‌ای شبیه README و خروجی نزدیک به GitHub-style Markdown.	<code>github</code>
جزوه‌های آموزشی طولانی و محتوای آموزشی فارسی/English.	<code>textbook</code>
گزارش رسمی، سند دانشگاهی، پیش‌نویس پایان‌نامه و مقاله ساختاریافته.	<code>academic</code>

حس رنگی	Palette
آبی حرفه‌ای پیش‌فرض.	blue
سبز آرام برای گزارش‌ها و داشبوردها.	emerald
بنفش خلاقانه برای سندهای محصولی.	violet
رنگ گرم برای محتوای آموزشی و review.	amber
رنگ برجسته برای گزارش‌های editorial.	rose
خنثی و رسمی برای مستندات فنی.	slate
خاکستری مینیمال برای خروجی رسمی.	neutral

انتخاب appearance از CLI:

BASH

```
folio input.md -o output.pdf --style modern --palette emerald --mode light
folio input.md -o output.pdf --style academic --palette emerald --mode dark
```

یا ذخیره در front matter:

YAML

```
appearance:
  style: modern
  palette: emerald
  mode: light
```

برای دیدن گزینه‌ها از `--list-styles`، `--list-palettes` و `--list-modes` استفاده کنید.

حالت تاریک برای هر style سطح رنگی مخصوص خودش را دارد. `modern` از پس‌زمینه سرمه‌ای عمیق استفاده می‌کند، `github` به سطح تاریک شبیه GitHub نزدیک است، `textbook` ظاهر تقریباً سیاه و هماهنگ با جلد قدیمی را نگه می‌دارد و `academic` پس‌زمینه ذغالی گرم دارد. Palette همچنان رنگ accent را در حالت روشن و تاریک، از جمله

پروفایل‌های کنترل کیفیت انتشار

برای کار روزمره و پیش‌نمایش تعاملی، پروفایل `standard` رفتار سازگار نسخه‌های قبلی را نگه می‌دارد: مشکل‌های کیفیت ثبت و به‌صورت warning اعلام می‌شوند، اما خروجی آزمایشی می‌تواند ادامه پیدا کند. برای پایان‌نامه، مقاله، گزارش عمومی یا artifact انتشار از `strict-publication` استفاده کنید تا اگر renderer نتواند کامل‌بودن فرمول‌ها، فونت‌های الزامی و navigation داخلی PDF را اثبات کند، build واقعاً fail شود.

BASH

```
folio report.md -o report.pdf \
  --quality-profile strict-publication \
  --require-font Vazirmatn \
  --require-font "JetBrains Mono" \
  --quality-report build/report-quality.json
```

گزارش تولیدشده JSON محدود و مناسب CI است. این فایل پروفایل انتخاب‌شده، تعداد pass/warning/error، تعداد فرمول‌های شناسایی و رندرشده MathJax، nodeهای حل‌نشده، در دسترس بودن فونت الزامی، font faceهای بارگذاری‌شده در مرورگر، hash فایل‌های فونت مورد اعتماد در `--font-dir`، مقصدهای PDF کپی یا بازسازی‌شده، بازنویسی لینک‌های داخلی و وضعیت نهایی artifact را ثبت می‌کند. خود فایل فونت یا متن کامل سند داخل گزارش قرار نمی‌گیرد.

رفتار هر گروه مستقل قابل override است:

BASH

```
folio report.md -o report.pdf \
  --quality-profile strict-publication \
  --math-error-policy error \
  --font-error-policy warn \
  --navigation-error-policy error
```

`error` ساخت را متوقف می‌کند، `warn` مشکل را ثبت کرده و ادامه می‌دهد و `ignore` ثبت می‌کند که شرط عمداً نادیده گرفته شده است. اگر policy صریح نوشته نشود، مقدار از profile می‌آید: برای `standard` برابر `warn` و برای `strict-publication` برابر `error`. در حالت strict اگر نام فونتی تعیین نشده باشد، `Vazirmatn` به صورت پیش‌فرض الزامی می‌شود. Mardas Folio فایل فونت را توزیع نمی‌کند؛ فونت مورد اعتماد را روی سیستم نصب کنید یا از `--font-dir` استفاده کنید.

همین تنظیم‌ها در `mardas.toml` نیز قابل ثبت هستند:

TOML

```
[quality]
profile = "strict-publication"
# های گروهی را ننویسید policy، error، برای به ارث بردن رفتار
required_fonts = ["Vazirmatn", "JetBrains Mono"]
report = "build/report-quality.json"
```

در Studio این گزینه‌ها داخل بخش **Publication Quality** قرار دارند. export صف‌شده پس از پایان یک خلاصه محدود از کیفیت برمی‌گرداند و خطای strict از همان مسیر خطای پایدار backend گزارش می‌شود. PDF-like preview مدرک نهایی نیست؛ PDF خروجی و quality report مربوط به آن ملاک انتشار هستند.

پیکربندی پروژه و Diagnostic های ساخت‌یافته

وقتی یک سند یا repository باید با تنظیمات ثابت و بدون فرمان طولانی ساخته شود، از فایل نسخه‌دار `mardas.toml` استفاده کنید. برای ایجاد فایل اولیه در پوشه فعلی بنویسید:

BASH

```
folio init
```

پیکربندی تولیدشده عمداً محافظه‌کارانه است: HTML امن، asset راه‌دور مسدود، صفحه A4 معمولی، appearance صریح و `schema_version = 1`. مسیر نسبی موجود در config نسبت به پوشه خود `mardas.toml` resolve می‌شود، نه نسبت به working directory ترمینال.

یک پیکربندی فشرده می‌تواند چنین باشد:

```
schema_version = 1
```

```
[project]
```

```
title = "گزارش روش‌های عددی"
```

```
author = "گروه پژوهشی"
```

```
direction = "auto"
```

```
[output]
```

```
page_size = "A4"
```

```
toc = true
```

```
toc_depth = 3
```

```
toc_page_break = true
```

```
cover = true
```

```
header_footer = true
```

```
mathjax = true
```

```
margin_top = "18mm"
```

```
margin_bottom = "20mm"
```

```
margin_x = "16mm"
```

```
[appearance]
```

```
style = "academic"
```

```
palette = "emerald"
```

```
mode = "light"
```

```
[branding]
```

```
mode = "off"
```

```
# logo = "assets/lab-logo.png"
```

```
[security]
```

```
unsafe_html = false
```

```
allow_remote_assets = false
```

```
[browser]
```

```
chromium_sandbox = "auto"
```

```
timeout_ms = 120000
```

```
[quality]
```

```
profile = "standard"
```

```
# math_errors = "warn"
```

```
# font_errors = "warn"
```

```
# navigation_errors = "warn"
```

```
# required_fonts = ["Vazirmatn"]
```

```
# report = "build/render-quality.json"
```

CLI از کنار فایل Markdown شروع می‌کند و نزدیک‌ترین `mardas.toml` را در مسیر پوشه‌های والد پیدا می‌کند. برای انتخاب صریح از `--config path/to/mardas.toml` و برای غیرفعال کردن discovery از `--no-config` استفاده کنید. ترتیب تقدم قطعی است:

TEXT

معادل < مقدار پیش‌فرض داخلی CLI > mardas.toml > front matter > گزینه صریح

گزینه‌های معکوس مثل `--no-toc`، `--cover`، `--header-footer`، `--mathjax`، `--safe-html` و `--block-remote-assets` اجازه می‌دهند مقدار فعال یا غیرفعال موجود در پروژه از داخل اسکریپت‌های automation override شود.

برای اعتبارسنجی config و Markdown بدون اجرای Chromium بنویسید:

BASH

```
folio validate report.md
folio validate report.md --format json
```

اعتبارسنجی، TOML یا YAML خراب، نسخه schema پشتیبانی‌نشده، کلید ناشناخته، مقدار نامعتبر، asset تنظیم‌شده ولی مفقود، تصویر block‌شده، پرش سطح heading و گزینه امنیتی پرریسک را گزارش می‌کند. Diagnostic‌ها شناسه پایدار مثل `MARDAS-E103`، `MARDAS-E109`، `MARDAS-W203` و `MARDAS-W301` دارند؛ automation باید به code تکیه کند، نه متن انگلیسی پیام.

برای مشاهده مقدار مؤثر و منبع هر تنظیم بنویسید:

BASH

```
folio explain-config report.md
folio explain-config report.md --format json
```

برای بررسی runtime محلی، MathJax بسته‌بندی‌شده، dependency‌های لازم، پیدا شدن Chromium، config و در صورت ارائه فایل، خود سند بنویسید:

BASH

```
folio doctor
folio doctor report.md --format json
```

Warning باعث fail شدن `validate` یا `doctor` نمی‌شود، اما diagnostic سطح error کد خروج غیرصفر تولید می‌کند. فعال‌کردن `unsafe_html` یا asset راه‌دور تصمیم صریح پروژه است و به شکل warning نمایش داده می‌شود، چون trust boundary را بزرگ‌تر می‌کند و می‌تواند reproducibility ساخت را کاهش دهد.

حالت کتاب چندفایلی

حالت Book Mode چند فایل Markdown را با ترتیب صریح و قطعی به یک PDF تبدیل می‌کند. این حالت برای کتاب، پایان‌نامه، گزارش بلند، جزوه درسی و مجموعه مستندات مناسب است که بهتر است source آن‌ها در فایل‌های کوچک‌تر و قابل مدیریت باقی بماند.

برای ساخت پروژه اولیه بنویسید:

BASH

```
folio init my-book --book
```

ساختار تولیدشده شامل `mardas.toml` و دو فصل نمونه است:

TEXT

```
my-book/  
├─ mardas.toml  
├─ chapters/  
│   ├─ 01-introduction.md  
│   └─ 02-content.md
```

در Book Mode ترتیب فصل‌ها فقط از آرایه مرتب `[book].chapters` گرفته می‌شود و به ترتیب نام فایل‌ها در filesystem وابسته نیست:

TOML

```
schema_version = 1

[project]
title = "راهنمای روش‌های عددی"
author = "گروه پژوهشی"
direction = "rtl"

[output]
toc = true
toc_depth = 4
cover = true
header_footer = true
mathjax = true

[book]
chapters = [
  "chapters/01-introduction.md",
  { path = "chapters/02-methods.md", title = "روش‌ها و آزمایش‌ها" },
  "chapters/03-results.md",
]
output = "dist/handbook.pdf"
chapter_page_break = true
```

فیلد اختیاری `title` عنوان سطح اول قابل مشاهده و عنوان همان فصل در فهرست کلی را عوض می‌کند، بدون اینکه فایل Markdown اصلی تغییر کند. اگر فصل heading سطح اول نداشته باشد، برنامه از عنوان فصل یک heading می‌سازد و diagnostic با کد `MARDAS-W501` گزارش می‌دهد.

برای اعتبارسنجی و مشاهده ساختار کامل کتاب بدون اجرای Chromium بنویسید:

BASH

```
folio validate-book my-book
folio validate-book my-book --format json
folio explain-book my-book --format json
```

برای ساخت PDF و در صورت نیاز ذخیره HTML ترکیبی: `renderer:`

```
folio build-book my-book
folio build-book my-book -o releases/handbook.pdf
folio build-book my-book --debug-html build/handbook.html --progress on
```

قواعد اصلی Book Mode عبارت‌اند از:

1. مسیر هر فصل باید نسبت به `mardas.toml` نوشته شود، پس از resolve شدن symlink داخل project root باقی بماند، پسوند Markdown پشتیبانی‌شده داشته باشد و فقط یک بار در manifest آمده باشد.
 2. تنظیمات پروژه، جلد، output، appearance، security، مرورگر، فونت، watermark و PDF مشترک را تعیین می‌کنند. Front matter هر فصل همچنان برای زبان و metadata محلی همان فصل استفاده می‌شود.
 3. تصویرها می‌توانند کنار هر فصل یا در پوشه مشترکی مانند `assets/` داخل project root قرار بگیرند. فایل بیرون از project root همچنان مسدود است.
 4. شناسه heading، anchor و footnote پیش از ترکیب برای هر فصل namespace می‌شود؛ بنابراین heading تکراری مانند `# مقدمه` بین فصل‌ها ابهام ایجاد نمی‌کند.
 5. لینک نسبی Markdown به فصل دیگری که در manifest وجود دارد، به لینک داخلی PDF تبدیل می‌شود. Fragment مانند `02-methods.md#model` به heading namespace همان فصل می‌رود. لینک فایل محلی دیگری برای قابل‌حمل ماندن PDF غیرفعال باقی می‌ماند.
 6. وقتی `chapter_page_break = true` باشد، بین فصل‌ها شکست صفحه درج می‌شود.
 7. خروجی نهایی یک جلد، یک فهرست کلی، یک outline تودرتوی PDF، یک مجموعه metadata، named destinationهای پایدار و یک فایل PDF دارد.
- حالت Book Mode از یک symbol table مشترک برای تمام فصل‌های manifest استفاده می‌کند. وقتی `numbering_scope = "chapter"` باشد، نخستین شیء فصل دوم شماره‌ای مثل `۲.۱` می‌گیرد؛ در حالت `global` شماره‌گذاری در کل کتاب پیوسته است. پردازش bibliography همچنان یک فاز مستقل است.

ارجاع متقابل و شماره‌گذاری

قابلیت ارجاع متقابل به صورت opt-in فعال می‌شود. آن را در `mardas.toml` یا front matter فعال کنید:

```
[references]
enabled = true
numbering_scope = "global"
list_of_figures = true
list_of_tables = true
list_of_equations = true
list_of_listings = true
```

همین گزینه‌ها را می‌توان در mapping با نام `references:` در front matter نوشت. در اتوماسیون خط فرمان نیز گزینه‌های `--references`، `--no-references`، `--numbering-scope global|chapter` و جفت گزینه‌های `--no-list-of-*` / `--list-of-*` قابل استفاده‌اند.

نوع‌های معنایی پشتیبانی‌شده عبارت‌اند از:

جدول ۱ نوع‌های معنایی ارجاع

پیشوند	شیء	محل label
<code>fig</code>	تصویر Markdown و شکل Mermaid	در caption یا marker مستقل بلافاصله بعد از شیء.
<code>tbl</code>	جدول Markdown	انتهای caption با شروع <code>Table:</code> .
<code>eq</code>	معادله نمایشی MathJax	marker مستقل بلافاصله بعد از معادله.
<code>lst</code>	code listing از نوع fenced	داخل metadata خط آغاز fence.

برای ارجاع به هر شیء، label معنایی تعریف کنید و از token متناظر `@kind:name` استفاده کنید:

را ببینید `@lst:training-loop` و کد `@eq:energy` معادله، `@tbl:metrics` جدول، `@fig:processing-flow` شکل

`!(معماری)(assets/architecture.svg)`

`{#fig:processing-flow}` شکل. معماری پردازش*

معیار	مقدار
دقت	۰/۹۸

Table: معیارهای ارزیابی `{#tbl:metrics}`

\$\$
E = mc^2
\$\$

`{#eq:energy}`

```
```python title="حلقه آموزش" {#lst:training-loop}
print("train")
```
```

نمونه‌های بعدی test case زنده renderer در همین راهنما هستند. ارجاع‌های شکل ۱، جدول ۱، معادله ۱ و کد ۱ باید در PDF به لینک‌های داخلی با شماره فارسی پایدار تبدیل شوند.

شکل ۱ جریان پردازش ارجاع



$$E = mc^2 \quad (۱)$$

کد ۱ اعتبارسنجی ارجاع

```
labels = {"fig:flow", "tbl:metrics"}
assert len(labels) == 2
```

Labelها باید در کل خروجی یکتا باشند. Book Mode پس از ترکیب همه فصلها ارجاعها را resolve می‌کند؛ بنابراین فصل اول می‌تواند به شیء labelدار در فصل بعدی اشاره کند. label تکراری، reference حل‌نشده، عدم تطابق نوع، label نامعتبر و marker جداافتاده پیش از شروع Chromium diagnosticهای پایدار `MARDAS-E60x` یا `MARDAS-W603` ایجاد می‌کنند. tokenهای داخل code، لینک موجود، script، style و contextهای literal تغییر نمی‌کنند.

فهرست‌های تولیدشده بعد از فهرست مطالب و پیش از بدنه سند قرار می‌گیرند و فقط شیءهایی را شامل می‌شوند که label صریح دارند. این راهنما هر چهار فهرست را به‌عنوان نمونه زنده release فعال می‌کند.

کتاب‌نامه و استناد

موتور استناد به‌صورت offline-first و opt-in طراحی شده است. این موتور فایل محلی BibTeX با پسوند `.bib` یا CSL JSON با پسوند `.json` را می‌خواند و هنگام build هیچ DOI lookup یا دریافت metadata از شبکه انجام نمی‌دهد.

فعال‌سازی در `mardas.toml`:

TOML

```
[bibliography]
enabled = true
sources = ["references.bib"]
style = "author-date"
include_uncited = false
# title = "منابع"
```

همین منبع را می‌توان در front matter تعریف کرد:

YAML

```
bibliography: references.bib
citations:
  enabled: true
  style: author-date
  include_uncited: false
```

شکل‌های پشتیبانی‌شده استناد عمداً کوچک و deterministic هستند:

MARKDOWN

[@doe2024] نتیجه با پژوهش پیشین سازگار است.
[@doe2024, p. 12; @smith2023]. چند منبع این موضوع را تأیید کرده‌اند.
روش را توضیح داده است @doe2024: استناد روایی

برای شماره‌گذاری بر اساس نخستین کاربرد از `style = "numeric"` استفاده کنید. حالت `author-date` استناد نویسنده/سال localized تولید می‌کند و وقتی یک نویسنده در یک سال چند اثر داشته باشد، پسوندهای deterministic مانند `a` و `b` را اضافه می‌کند. هر مدخل کتاب‌نامه به نخستین استناد back-link دارد و هر استناد به destination پایدار `#bib-*` متصل می‌شود. در Book Mode منابع پروژه فقط یک بار بارگذاری می‌شوند و استنادها پس از ترکیب همه فصل‌ها resolve می‌شوند؛ بنابراین کل کتاب فقط یک کتاب‌نامه دارد.

جمله بعدی test case زنده renderer است. باید دو استناد گروهی تفکیک‌شده (**Rastegar, ۲۰۲۶a, p. 14**)؛ (**Rastegar, ۲۰۲۶b**) و یک استناد روایی به **احمدی (۱۴۰۴)** تولید کند و در انتهای همین راهنما فقط یک بخش منابع ساخته شود.

Validation، منبع مفقود، BibTeX یا CSL JSON نامعتبر، کلید تکراری، citation حل‌نشده، گروه citation خراب، source تکراری، فایل بیش‌ازحد بزرگ و تعداد مدخل بیش‌ازحد را با diagnostic‌های پایدار `MARDAS-E70x` پیش از اجرای Chromium رد می‌کند. متن شبیه citation داخل code، لینک موجود، script، style و context‌های literal بدون تغییر باقی می‌ماند.

کارایی و اسناد بزرگ

خروجی‌های تکراری Studio از یک صف محدود و نشست پایدار renderer استفاده می‌کنند. هر worker می‌تواند یک فرایند Chromium را دوباره به‌کار گیرد، اما برای هر خروجی یک browser context تازه ساخته می‌شود تا وضعیت صفحه، cookie، object URL و storage میان سندها منتقل نشود. نوار پایین Studio مرحله واقعی رندر و درصد پیشرفت را نمایش می‌دهد. دکمه **Cancel** کار در صف را فوراً لغو می‌کند و برای کار در حال اجرا درخواست لغو تعاملی می‌فرستد.

برای تنظیم صف محلی، Studio را به شکل زیر اجرا کنید:

BASH

```
folio-gui \  
  --render-workers 2 \  
  --export-queue-size 6 \  
  --render-idle-timeout 60
```

گزینه `--render-workers` تعداد workerهای هم‌زمان Chromium، گزینه `--export-queue-size` حداکثر کارهای منتظر و گزینه `--render-idle-timeout` زمان بسته‌شدن browser بدون استفاده را تعیین می‌کند. افزایش worker ممکن است throughput را بیشتر کند، اما peak memory را نیز بالا می‌برد؛ بدون benchmark مقدار پیش‌فرض را تغییر ندهید.

برای اندازه‌گیری بازتولیدپذیر قبل و بعد از تغییرات اجرا کنید:

BASH

```
python scripts/benchmark_large_documents.py \  
  --profiles small,pages50,pages250,pages500,editor-loop \  
  --mode both \  
  --repeats 3 \  
  --output-dir build/performance
```

حالت `cold` برای هر تبدیل Chromium را از نو اجرا می‌کند. حالت `session` Chromium را نگه می‌دارد و برای هر تکرار browser context جدید می‌سازد. در یک محیط یکسان، توزیع زمان اجرا، تعداد صفحه، اندازه PDF، تعداد اجرای browser و peak RSS گزارش JSON را مقایسه کنید. یک اجرای سریع به‌تنهایی مدرک کافی برای بهبود کارایی نیست.

PDFهای تکمیل‌شده Studio فقط تا زمان دانلود یا انقضا در فضای موقت خصوصی و محدود نگهداری و به‌صورت streaming ارسال می‌شوند. لغو کار تعاملی است؛ اگر `page.pdf()` در Chromium در حال اجرا باشد، آن فراخوانی تا checkpoint امن بعدی کامل می‌شود.

دسترس‌پذیری و آمادگی آرشیوی

زبان سند را به‌صورت صریح مشخص کنید تا HTML، metadata فایل PDF، ابزارهای کمکی و برچسب‌های محلی‌سازی‌شده سیگنال زبانی پایداری دریافت کنند:

TOML

```
[project]
language = "fa-IR"
direction = "rtl"
```

همین مقدار را می‌توان در front matter با `lang: fa-IR` یا برای یک تبدیل با `--lang fa-IR` تعیین کرد.

پیش از اجرای Chromium، audit منبع را اجرا کنید:

BASH

```
folio audit-accessibility report.md
folio audit-accessibility report.md --format json --fail-on warning
folio audit-book-accessibility path/to/book
```

این audit ساختار headingها، متن جایگزین تصویر، نام لینکها، header و caption جدول، زبان اعلام‌شده و کنتراست appearance انتخاب‌شده را بررسی می‌کند. خطاها و هشدارها با کدهای پایدار `MARDAS-A...` و در صورت امکان با نام فایل و شماره خط گزارش می‌شوند.

از متن توصیفی و ساختار معنایی جدول استفاده کنید:

MARKDOWN

(images/architecture.png)[PDF به خروجی Markdown جریان تبدیل ورودی]!

.شکل. نمای کلی فرایند انتشار

| مرحله | خروجی قابل بررسی |

| :--- | :--- |

| ساخت‌یافته Diagnostic | منبع Audit |

| و آمادگی فونت metadata گزارش | PDF فایل Audit |

جدول: مرحله‌های ارزیابی دسترس‌پذیری

را مطالعه کنید (../SECURITY.md)[راهنمای امنیت و دسترس‌پذیری]

در HTML نهایی، شکل‌های تصویری به caption خود متصل می‌شوند، caption جدول شناسه پایدار می‌گیرد، headerهای سطر و ستون `scope` دریافت می‌کنند و جدول به caption خود ارجاع می‌دهد. این بهبودهای HTML به‌تنهایی به معنای tagged PDF یا انطباق PDF/UA نیستند.

فایل نهایی را نیز جداگانه بررسی کنید:

BASH

```
folio audit-pdf output.pdf --profile accessibility
folio audit-pdf output.pdf --profile archival
folio audit-pdf output.pdf --profile all --format json --fail-on never
```

PDFهای تولیدشده `/Lang` در catalog، تنظیم نمایش عنوان، metadata اطلاعات سند و XMP metadata دارند. audit فایل PDF وضعیت embedding فونت‌ها، ToUnicode، tagging، output intent، JavaScript، attachment و شناسه‌های PDF/A را گزارش می‌کند. خروجی فعلی Chromium به‌عنوان PDF/UA compliant معرفی نمی‌شود و پروژه structure tree یا شناسه PDF/A جعلی اضافه نمی‌کند. انطباق رسمی به validator مستقل و بازبینی دستی دسترس‌پذیری نیاز دارد.

روند کار با GUI

اجرای GUI برای یک سند مستقل:

BASH

```
folio-gui
```

باز کردن workspace واقعی پروژه بر پایه `mardas.toml`:

BASH

```
folio-gui --project path/to/project
```

در حالت Project Workspace، فصل‌های مرتب‌شده Book Mode و فایل‌های متنی پشتیبانی‌شده پروژه در Project Explorer نمایش داده می‌شوند. Problems Panel همان diagnosticهای پایدار `MARDAS - *` در فرمان‌های `validate` و `validate-book` را نشان می‌دهد؛ انتخاب هر مشکل، فایل مسئول را باز می‌کند و در صورت وجود

اطلاعات مکان، editor را به خط و ستون مربوط می‌برد. Preview فایل Markdown فعال با تنظیمات واقعی پروژه، کتاب‌نامه، cross-reference و مرز asset های project root ساخته می‌شود. دو فرمان **Preview Book** و **Export Book** کل نسخه ذخیره‌شده پروژه را پردازش می‌کنند؛ بنابراین ابتدا فایل فعال را ذخیره کنید.

ویرایش فایل واقعی فقط زمانی فعال می‌شود که Studio صریحاً با `--project` اجرا شده باشد. Studio فقط فایل متنی UTF-8 پشتیبانی‌شده داخل project root را می‌پذیرد، مسیر hidden یا generated و symlink را رد می‌کند، برای هر فایل سقف ۴ مگابایت دارد و با SHA-256 نسخه بازشده، تغییر خارجی فایل را هنگام ذخیره تشخیص می‌دهد. ذخیره موفق به صورت atomic انجام می‌شود و permission فعلی فایل را حفظ می‌کند. کنترل‌های **Open Bundle** و **Save Bundle** همچنان برای snapshot قابل حمل `.mardas.json` در دسترس‌اند؛ bundle با workspace زنده پروژه یکسان نیست.

رابط کاربری GUI برای کاربرانی مناسب است که روند بصری را ترجیح می‌دهند:

1. نوشتن یا paste کردن. Markdown.
 2. تکمیل بخش **Document** برای عنوان، نویسنده، نام فایل خروجی، اندازه صفحه و جهت.
 3. انتخاب کارتهای **Appearance** برای style، palette و mode روشن/تاریک.
 4. استفاده از **Branding** فقط وقتی PDF باید نشان سازمان، محصول یا آزمایشگاه داشته باشد.
 5. استفاده از **Layout** برای فهرست مطالب، جلد و جریان صفحه‌ها.
 6. باز کردن **Advanced** فقط برای watermark، حذف شماره صفحه یا attach کردن asset های محلی.
 7. استفاده از preview پیش‌فرض PDF-like برای HTML تولیدشده توسط renderer با شبیه‌سازی اندازه صفحه، margin و scale شدن صفحه؛ Fast Preview فقط برای بازخورد بسیار سریع هنگام ویرایش است و parsing آن تقریبی است. Fast Preview مسیر تصویر remote/local را fetch نمی‌کند و scheme های ناامن یا فایل سیستمی لینک را غیرفعال می‌کند.
 8. استفاده از **S+Ctrl/Cmd** برای ذخیره Markdown یا فایل فعال Project Workspace، **S+Shift+Ctrl/Cmd** برای ذخیره bundle قابل حمل `.mardas.json` و **Enter+Ctrl/Cmd** برای export سریع PDF.
- Studio پیش‌نویس فعلی، layout، حالت روشن/تاریک، جهت preview، عرض editor و گزینه‌های export را در local storage مرورگر نگه می‌دارد. این کار باعث می‌شود refresh ناخواسته صفحه در جلسه‌های ویرایش طولانی کمتر آزاردهنده باشد. برای پاک کردن پیش‌نویس محلی و برگشتن به حالت تمیز از **Reset State** استفاده کنید. همین بخش مرجع روند کار با Studio برای کاربر است.

اگر export با خطا روبه‌رو شود، Studio وضعیت HTTP و کد پایدار backend مثل `invalid_json`، `markdown_too_large`، `invalid_watermark_opacity`، `invalid_toc_depth`، `invalid_page_size` یا `render_failed` را نشان می‌دهد. اگر GUI را روی host غیرلوکال bind کنید، backend هشدار می‌دهد؛ چون کاربران قابل دسترس در شبکه می‌توانند Markdown و asset بفرستند.

Studio به صورت پیش فرض preview نوع PDF-like با scale خودکار صفحه، اندازه صفحه و margin را نشان می دهد. Fast Preview برای ویرایش فوری مفید است، اما parser محلی مرورگر آن تقریبی و URL policy آن عمداً محدود است؛ fidelity نهایی، محل شکست صفحه، assetهای متصل، MathJax، Mermaid و validation امنیتی هنگام export توسط backend renderer و layout چاپی Chromium تعیین می شوند.

مرجع CLI

| توضیح | گزینه |
|---|---|
| فایل Markdown ورودی. | <code>input</code> |
| مسیر PDF خروجی. | <code>-o</code> , <code>--output</code> |
| override کردن metadata موجود در front matter. | <code>--title</code> , <code>--author</code> , <code>--description</code> |
| فعال سازی و تنظیم فهرست مطالب. | <code>--toc</code> , <code>--toc-depth</code> |
| فعال یا غیرفعال کردن شماره گذاری و ارجاع معنایی. | <code>--references</code> , <code>--no-references</code> |
| انتخاب شماره گذاری پیوسته <code>global</code> یا شماره گذاری فصل محور <code>chapter</code> در Book Mode. | <code>--numbering-scope</code> |
| ساخت فهرست های انتخابی؛ هر گزینه جفت <code>--no-list-of-*</code> نیز دارد. | <code>--list-of-figures</code> , <code>--list-of-tables</code> , <code>--list-of-equations</code> , <code>--list-of-listings</code> |
| فعال یا غیرفعال کردن resolve استناد و ساخت کتاب نامه. | <code>--citations</code> , <code>--no-citations</code> |
| افزودن منبع محلی <code>.bib</code> یا CSL <code>.json</code> ؛ برای چند منبع تکرار شود. | <code>--bibliography PATH</code> |
| انتخاب style داخلی <code>author-date</code> یا <code>numeric</code> . | <code>--citation-style</code> |
| تنظیم عنوان کتاب نامه و حضور یا حذف مدخل های استناد نشده. | <code>--bibliography-title</code> , <code>--include-uncited</code> , <code>--cited-only</code> |
| کنترل صفحه بندی چاپی. | <code>--toc-page-break</code> , <code>--h1-page-break</code> |
| انتخاب <code>academic</code> یا <code>textbook</code> , <code>github</code> , <code>modern</code> . | <code>--style</code> |
| انتخاب <code>rose</code> , <code>amber</code> , <code>violet</code> , <code>emerald</code> , <code>blue</code> یا <code>slate</code> , <code>neutral</code> . | <code>--palette</code> |
| انتخاب <code>light</code> یا <code>dark</code> . | <code>--mode</code> |
| نمایش گزینه های appearance و خروج. | <code>--list-styles</code> , <code>--list-palettes</code> , <code>--list-modes</code> |
| اندازه صفحه مثل <code>A4</code> , <code>landscape</code> , <code>Legal</code> , <code>Letter</code> یا ابعادی مثل <code>210mm</code> , <code>297mm</code> . | <code>--page-size</code> |
| اجبار جهت به <code>ltr</code> , <code>auto</code> یا <code>rtl</code> . | <code>--dir</code> |
| کنترل margin صفحه. | <code>--margin-top</code> , <code>--margin-bottom</code> , <code>--margin-x</code> |
| مسیر فونت های محلی Vazirmatn. | <code>--font-dir</code> |

| گزینه | توضیح |
|--|---|
| <code>--chromium-path</code> | مسیر سفارشی Chromium یا Chrome. |
| <code>--chromium-sandbox</code> | حالت sandbox مرورگر: <code>auto</code> ، <code>on</code> یا <code>off</code> . مقدار پیش‌فرض: <code>auto</code> . |
| <code>--debug-html</code> | ذخیره HTML میانی. |
| <code>--no-cover</code> ، <code>--branding</code> ، <code>--brand-name</code> ، <code>--brand-logo</code> ، <code>--brand-footer</code> ، <code>--no-cover-logo</code> | تنظیم جلد و برندینگ صریح. |
| <code>--watermark</code> ، <code>--watermark-image</code> | افزودن watermark متنی یا تصویری با لایه‌بندی هماهنگ با mode. |
| <code>--allow-remote-assets</code> | اجازه به سندهای قابل اعتماد برای بارگذاری تصاویرهای remote <code>http(s)</code> . پیش‌فرض غیرفعال است. |
| <code>--no-header-footer</code> | حذف footer چاپی. |
| <code>--no-mathjax</code> | غیرفعال کردن MathJax. |
| <code>--unsafe-html</code> | غیرفعال کردن sanitization برای فایل‌های قابل اعتماد. |
| <code>--timeout-ms</code> | timeout مرورگر بر حسب میلی‌ثانیه. |
| <code>--progress</code> | حالت progress bar ترمینال: <code>auto</code> ، <code>on</code> یا <code>off</code> . مقدار پیش‌فرض: <code>auto</code> . |

برای دیدن همه گزینه‌ها:

BASH

```
folio --help
```

اتوماسیون و CI

یک دستور ساده برای ساخت PDF در اسکریپت:

BASH

```
folio docs/report.md -o build/report.pdf --toc --style modern --palette emerald --mode light
```

مخزن شامل workflow گیت‌هاب Actions است که Ruff، pytest و یک smoke test واقعی رندر با Chromium را روی نسخه‌های پشتیبانی‌شده Python اجرا می‌کند. برای CI بهتر است گزینه‌ها صریح باشند:

BASH

```
folio docs/report.md -o build/report.pdf \  
  --toc \  
  --toc-depth 4 \  
  --style modern \  
  --palette emerald \  
  --mode light \  
  --page-size A4 \  
  --dir auto \  
  --timeout-ms 60000 \  
  --progress off
```

برای اشکال‌زدایی در CI، HTML میانی را به عنوان artifact نگه دارید:

BASH

```
folio docs/report.md -o build/report.pdf --debug-html build/report.html
```

■ اعتبارسنجی فایل‌های Release

خروجی رسمی release شامل wheel بازتولیدپذیر، source distribution، PDF راهنماهای فارسی و English، فایل SBOM با قالب SPDX 2.3، فایل `RELEASE-MANIFEST.json` و فایل `CHECKSUMS.sha256` است. بسته‌های نصب آفلاین Python برای Linux، Windows و macOS به صورت جداگانه ساخته می‌شوند.

قبل از نصب، checksumها را بررسی کنید:

BASH

```
sha256sum -c CHECKSUMS.sha256
```

برای کنترل manifest و ساختار SBOM از داخل checkout پروژه استفاده کنید:

BASH

```
python scripts/finalize_release_artifacts.py \  
  --artifact-dir path/to/release \  
  --version X.Y.Z \  
  --require-sbom \  
  --verify-only
```

فایل دریافت‌شده از GitHub را می‌توان در برابر provenance امضا شده نیز بررسی کرد:

BASH

```
gh attestation verify artifact-name \  
  --repo mragetsars/Mardas-Folio
```

پس از استخراج بسته آفلاین Python، فرمان زیر را اجرا کنید:

BASH

```
python install.py --target mardas-folio-venv
```

این بسته wheel‌های Python را بدون تماس با package index نصب می‌کند. Python و Chromium مستقل داخل آن قرار ندارند؛ نصب browser یک مرحله جداگانه و صریح است.

رفع اشکال

پیدا نشدن Chromium

این دستور را اجرا کنید:

BASH

```
python -m playwright install chromium
```

یا مسیر مرورگر را صریح بدهید:

BASH

```
folio input.md -o output.pdf --chromium-path /path/to/chrome
```

■ متن فارسی ظاهر مناسبی ندارد

یک فونت فارسی مناسب مثل Vazirmatn نصب کنید یا مسیر فونت را بدهید:

BASH

```
folio input.md -o output.pdf --font-dir ./fonts
```

اگر مسیر فونت وجود نداشته باشد یا فایل شناخته‌شده‌ای داخل آن پیدا نشود، renderer هشدار می‌دهد و از فونت‌های سیستم استفاده می‌کند.

■ تصویرها دیده نمی‌شوند

مطمئن شوید مسیر تصویرها نسبت به فایل Markdown درست است و داخل پوشه همان سند باقی می‌ماند. مسیرهای absolute، URL‌های `file:`، خروج از پوشه با `...`، فایل‌های گم‌شده و تصویرهای خیلی بزرگ با placeholder امن جایگزین می‌شوند تا Chromium آن‌ها را از فایل سیستم load نکند. اگر از GUI استفاده می‌کنید، فایل‌ها یا پوشه‌های تصویر را قبل از خروجی گرفتن attach کنید و هشدارهای renderer را بررسی کنید.

■ فرمول‌ها به شکل TeX خام دیده می‌شوند

مطمئن شوید MathJax فعال است و هشدارهای renderer را بررسی کنید. برای سندهای بزرگ مقدار timeout را افزایش دهید:

BASH

```
folio input.md -o output.pdf --timeout-ms 90000
```

■ نیاز به بررسی Layout

فایل HTML میانی را ذخیره کنید:

BASH

```
folio input.md -o output.pdf --debug-html output.html
```

سپس `output.html` را در مرورگر باز کنید و ساختار و CSS تولیدشده را بررسی کنید.

بررسی Preflight فایل PDF

برای PDFهای مهم و عمومی، بعد از ساخت exampleها یک preflight سریع اجرا کنید. این بررسی چند صفحه نماینده را raster می‌کند، فونت‌های embed شده را فهرست می‌کند و هشدارهای syntax مربوط به PDF را قبل از انتشار ثبت می‌کند:

BASH

```
python scripts/check_pdf_preflight.py \  
  examples/GUIDE.en.pdf \  
  examples/GUIDE.fa.pdf \  
  --pages 1,2,3 \  
  --output build/pdf-preflight.json
```

خروجی تصویری تمیز همچنان معیار اصلی است، اما گزارش preflight محل تکرارپذیری برای گرفتن regressionهای مربوط به فونت، rasterization و parserهای PDF فراهم می‌کند.

پانویس

پانویس برای ارجاع، یادداشت فنی و توضیح تکمیلی مناسب است.^۲

۲. برنامه Mardas Folio به‌جای رسم مستقیم هر پاراگراف روی canvas فایل PDF، از Chromium برای layout استفاده می‌کند. این انتخاب باعث پشتیبانی بهتر از CSS print، متن ترکیبی، خروجی SVG MathJax، جدول‌ها، تصویرهای محلی و کدهای هایلایت‌شده می‌شود.
- بلوک پانویس نزدیک همان ارجاع درج می‌شود و دیگر به انتهای سند منتقل نمی‌شود.
 - پانویس چندخطی پشتیبانی می‌شود.
 - متن Markdown داخل پانویس حفظ می‌شود.
 - توجه کنید که inline code مثل `@page`، `$x$` و `[^id]` وقتی داخل backtick نوشته شود خوانا و literal باقی می‌ماند.

چک لیست نهایی انتشار

قبل از انتشار PDF مهم، این موارد را بررسی کنید:

- ☒ عنوان جلد، زیرعنوان، نویسنده، تاریخ و metadata.
- ☒ زبان و جهت سند.
- ☒ عمق و لیبل‌های فهرست مطالب.
- ☒ صفحه‌های دارای فرمول.
- ☒ صفحه‌های دارای code block و inline code.
- ☒ صفحه‌های دارای تصویر محلی.
- ☒ جدول‌هایی که محتوای پهن دارند.
- ☒ پانویس‌ها و لینک‌ها.
- ☒ شماره‌گذاری footer بعد از جلد.
- ☐ بازبینی بصری نهایی در PDF viewer.
- ☐ هنگام انتشار نسخه tag شده، چک لیست release در `docs/RELEASE.md` کامل شده باشد.

Rastegar, Meraj. (۲۰۲۶a). **Deterministic Publishing for Mixed-Direction Documents**. *Mardas Publishing Notes*. 1, (1), 1–18. [doi:10.5555/mardas.2026.1](https://doi.org/10.5555/mardas.2026.1). ↩

Rastegar, Meraj. (۲۰۲۶b). **Offline Citation Pipelines for Reproducible PDF Output**. *Mardas Publishing Notes*. 1, (2), 19–34. ↩

احمدی، مریم. (۱۴۰۴). انتشار علمی فارسی و مدیریت منابع. انتشارات پژوهش. ↩